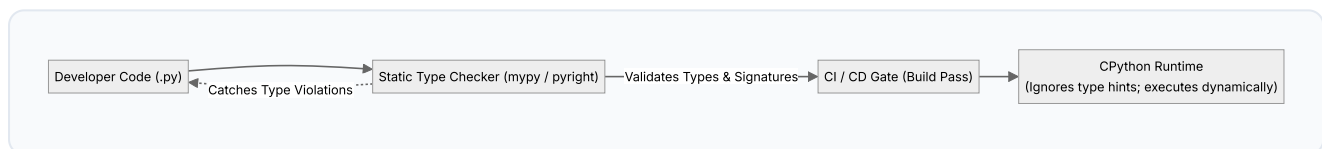


Chapter 07: Type Systems, Protocols & Modern Python Architecture

Core Learning Objective: Write robust, self-documenting, production-grade Python. Master gradual typing, covariant/contravariant generics, structural subtyping via `typing.Protocol`, Pydantic v2 data validation, and clean architectural separation.

1. The Modern Python Type System

Python remains a dynamically typed runtime, but large-scale enterprise codebases rely on **Static Type Analysis** (`mypy`, `pyright`) to eliminate entire categories of production bugs before deployment.



Essential Typing Primitives (Python 3.10+)

2. Generics & Variance: Covariance vs Contravariance

Generics enable you to write reusable algorithms while preserving type safety across diverse data types.

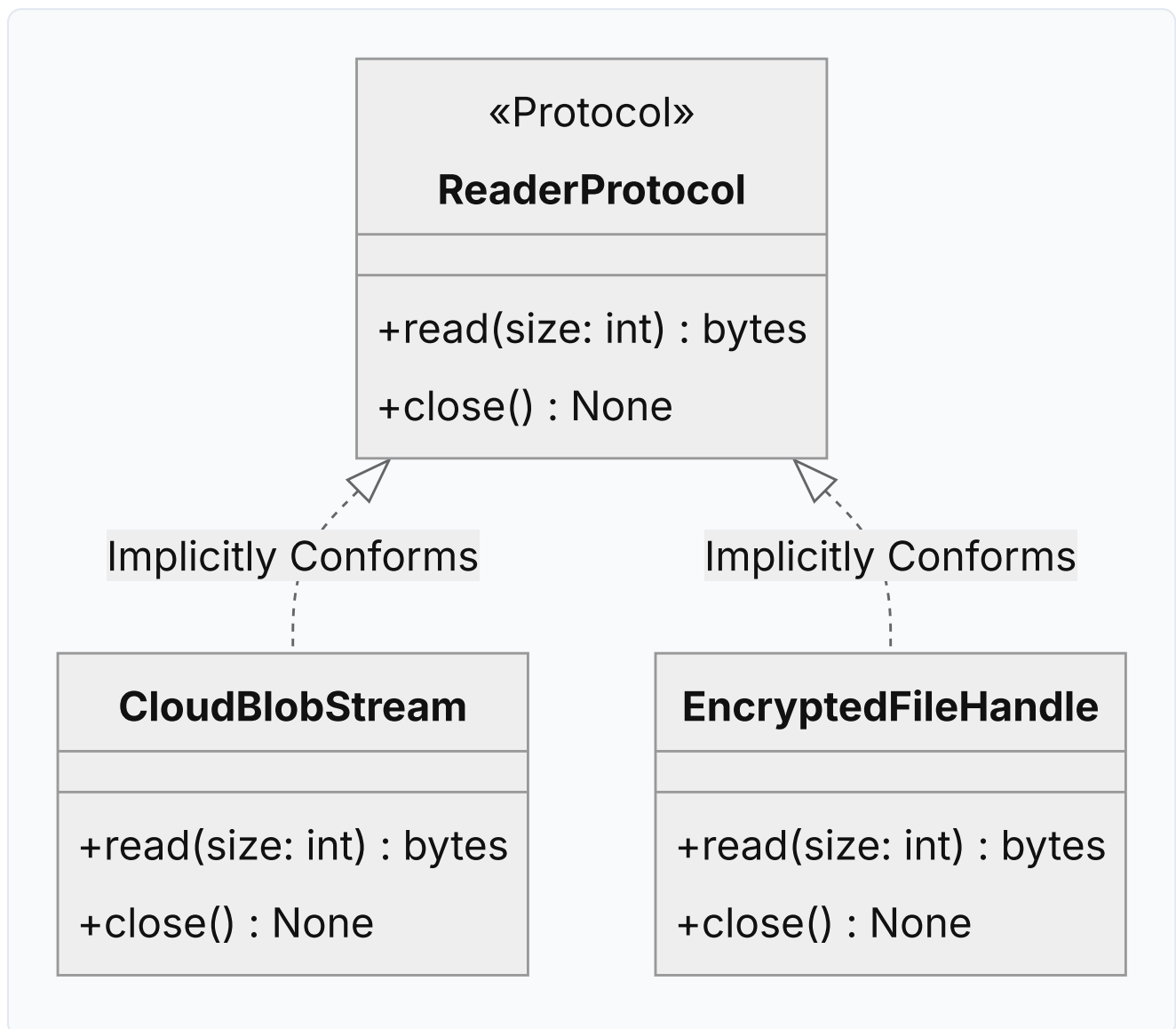
Variance Explained: Covariant vs Contravariant

- **Invariant** (`TypeVar('T')`): A `Container[Dog]` is **not** a `Container[Animal]`, even if `Dog` inherits from `Animal`. Prevents adding an `Animal` (like a `Cat`) into a dog container.
- **Covariant** (`TypeVar('T_co', covariant=True)`): Read-only producers. If `Dog` is an `Animal`, then `Sequence[Dog]` can be safely substituted where `Sequence[Animal]` is expected.
- **Contravariant** (`TypeVar('T_contra', contravariant=True)`): Write-only consumers. If `Dog` is an `Animal`, a handler that accepts any `Animal` can safely process a `Dog`.

3. Structural Subtyping: `typing.Protocol` (Formalized Duck Typing)

In classic OOP, a class must explicitly inherit from an Abstract Base Class (`class SqlDb(DatabaseABC): ...`) to be recognized as satisfying an interface (**Nominal Subtyping**).

Structural Subtyping (`Protocol`) decouples implementations: any class that possesses the required methods and attributes automatically satisfies the Protocol, **with zero inheritance needed**:



4. Modern Data Models: Dataclasses vs Pydantic v2

Python Standard `dataclass` (Best for internal domain models)

Pydantic v2 (Best for network boundaries, JSON APIs, and Agentic AI)

Pydantic v2 is written in Rust (`pydantic-core`). It performs **high-speed runtime data validation, coercion, and serialization**:

5. Enterprise Exception Hierarchies & Exception Groups

1. Unified Exception Trees

Never raise generic `Exception` or `RuntimeError` in production libraries. Define a domain-specific hierarchy:

2. Exception Groups & `except*` (Python 3.11+)

When multiple concurrent tasks crash inside `asyncio.TaskGroup`, multiple independent exceptions occur simultaneously. Python handles this with **Exception Groups**:

6. Production Repository Structure (12-Factor Ready)

Version Notes

- `X | Y` unions need 3.10 (PEP 604); `typing.Self` 3.11; `typing.override` 3.12; the `type` statement and `class Box[T]:` syntax 3.12 (PEP 695); TypeVar defaults 3.13 (PEP 696).
- Annotations are evaluated lazily by default from 3.14 (PEP 649/749), which makes `from __future__ import annotations` unnecessary there; on older versions it is still the way to

avoid forward-reference errors.

- `dataclass(slots=True, kw_only=True)` need 3.10.

Exercises

Three graded exercises for this chapter (two coding, one debugging) with hidden tests:



Attempt first; the reference solutions are in `exercises/_answers/ch07.py`.

Further Reading

- [typing module](#)
- [mypy documentation](#)
- [PEP 544: Protocols](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. Does the interpreter enforce type hints at runtime?
- ▶ 2. What does `Protocol` give you that an abstract base class does not?
- ▶ 3. When would you use a `dataclass(frozen=True, slots=True)`?
- ▶ 4. What is `TypeVar` for?